

COMPTE RENDU DE PROJET eVoting — Plateforme de vote électronique sécurisée

1. Contexte et objectifs du projet

Le projet eVoting a pour but de concevoir un système de vote électronique s'inspirant des principes de la blockchain (immutabilité, traçabilité, vérifiabilité des votes) sans recourir à une véritable infrastructure décentralisée. L'objectif est de démontrer, à travers un backend fonctionnel, comment une élection peut être organisée numériquement de façon sécurisée : enregistrement des électeurs, ajout de candidats, ouverture et clôture du scrutin, émission de votes, puis vérification de leur intégrité grâce à un système de hachage cryptographique.

Le projet illustre également, à travers un module de benchmark dédié, l'intérêt du calcul parallèle (GPU) par rapport au calcul séquentiel (CPU) pour des opérations de hachage intensives, ce qui fait écho aux problématiques de performance rencontrées dans les véritables réseaux blockchain (minage, validation de blocs).

2. Technologies utilisées

Langage	Python 3
Framework backend	Flask + Flask-CORS
Exposition publique	pyngrok (tunnel ngrok)
Sécurité / intégrité	hashlib (SHA-256)
Calcul accéléré	PyTorch (CUDA) pour le benchmark GPU
Environnement d'exécution	Google Colab (GPU Tesla T4)
Concurrence	threading (serveur Flask lancé en tâche de fond)

3. Architecture du système

Le système repose sur un modèle d'état centralisé en mémoire (dictionnaire Python state) qui simule le registre d'une blockchain :

- admin / contract : adresses simulées de l'administrateur et du "smart contract"

- candidates : liste des candidats à l'élection, avec compteur de voix
- voters : registre des électeurs (statut d'inscription, statut de vote, hash du vote)
- votes : journal des votes horodatés et hachés
- validators : adresses simulant des nœuds validateurs du réseau

Chaque vote génère une empreinte SHA-256 calculée à partir de l'adresse de l'électeur, de l'identifiant du candidat, de l'horodatage et de l'adresse du contrat. Cette empreinte permet ensuite de vérifier a posteriori qu'un vote n'a pas été altéré (fonction validate-votes).

Le serveur Flask est lancé dans un thread dédié, puis rendu accessible depuis l'extérieur du notebook Colab grâce à un tunnel ngrok, ce qui permet à un frontend externe (application web) de consommer l'API en temps réel.

4. Fonctionnalités développées (API REST)

Méthode	Route	Rôle
GET	/api/status	État général de l'élection (démarrée, terminée, nombre de votes...)
GET	/api/accounts	Liste des comptes simulés disponibles
GET	/api/candidates	Liste des candidats enregistrés
POST	/api/add-candidate	Ajout d'un candidat avant le début du scrutin
POST	/api/register-voter	Inscription d'un électeur par adresse
POST	/api/start-election	Démarrage officiel de l'élection
POST	/api/end-election	Clôture de l'élection
POST	/api/vote	Émission d'un vote (avec vérifications d'unicité et d'éligibilité)
GET	/api/has-voted	Vérifie si un électeur a déjà voté
GET	/api/verify-vote	Vérifie la validité d'un vote via son hash

GET	/api/winner	Détermine le candidat vainqueur une fois l'élection close
POST	/api/validate-votes	Revalide l'intégrité de tous les votes enregistrés
GET	/api/benchmark	Compare les performances de hachage CPU vs GPU

5. Déroulement et résultats des tests

Le notebook a été exécuté avec succès sur un runtime Google Colab disposant d'un GPU Tesla T4. Les étapes suivantes ont été validées :

- Installation des dépendances (Flask, Flask-CORS, pyngrok) sans erreur
- Détection automatique du GPU (CUDA — Tesla T4)
- Initialisation de la blockchain simulée avec génération de l'administrateur et de 9 comptes électeurs
- Démarrage du serveur Flask sur le port 5000 et exposition via ngrok
- Test automatique de l'API : vérification du statut, puis ajout de 3 candidats de démonstration (Alice Martin, Bob Dupont, Clara Benali)
- Correction en cours de session : l'administrateur, initialement non enregistré comme électeur, a été ajouté manuellement au registre des votants pour pouvoir participer au scrutin

L'URL publique générée par ngrok a été affichée avec succès, confirmant que le backend était prêt à être connecté à un frontend externe.

6. Difficultés rencontrées et solutions apportées

- Oubli initial : l'administrateur n'était pas inscrit comme électeur → corrigé via une cellule de débogage ajoutant manuellement son adresse au registre voters.
- Nécessité de vérifier la correspondance entre les adresses générées (accounts) et celles enregistrées dans voters, résolue par l'ajout d'une cellule d'inspection dédiée.
- Gestion de l'exposition publique du serveur : utilisation de ngrok pour contourner les limitations réseau de l'environnement Colab.

7. Limites identifiées

- La "blockchain" est entièrement simulée en mémoire : elle n'offre ni décentralisation, ni consensus distribué, ni persistance des données après l'arrêt du runtime.
- Absence de base de données : toutes les informations (votes, candidats, électeurs) sont perdues au redémarrage du notebook.

- Le jeton d'authentification ngrok est actuellement codé en dur dans le notebook, ce qui constitue une mauvaise pratique de sécurité à corriger (utilisation d'une variable d'environnement recommandée).
- Absence de verrouillage (lock) sur les structures de données partagées, ce qui pourrait poser des problèmes en cas de requêtes concurrentes à grande échelle.
- Le code principal est concentré dans une seule cellule volumineuse, ce qui limite la lisibilité et la maintenabilité du notebook.

8. Conclusion et perspectives

Ce projet a permis de mettre en œuvre un backend complet de vote électronique, illustrant les concepts clés d'une blockchain (hachage, immuabilité, vérifiabilité) tout en restant accessible à mettre en place rapidement dans un environnement Colab. Le module de benchmark CPU/GPU apporte une dimension supplémentaire en illustrant concrètement l'intérêt du calcul parallèle pour des opérations cryptographiques répétitives.

Les prochaines étapes envisageables seraient : la mise en place d'une véritable persistance des données (base de données), l'ajout d'une couche de sécurité (authentification, chiffrement du jeton ngrok), ainsi qu'un découplage du code en modules plus lisibles pour faciliter la maintenance et l'évolution du projet.